Checklist Final para Entrega do Projeto

A estrutura principal do seu sistema está de pé. Agora, o foco é implementar as funcionalidades-chave que cumprem todos os requisitos do trabalho.

✅ Tarefas Críticas de Back-End

Estas são as funcionalidades que precisam ser implementadas no seu código Java (Spring Boot) para que o front-end tenha o que consumir.

1. Implementar Segurança Real (Requisito RNF03)

- **O que falta:** Atualmente, o login é simulado e as senhas não são seguras.
- **Ações:**
  - **Adicionar Spring Security:** Incluir a dependência de segurança no pom.xml.
  - **Criptografar Senhas:** Usar BCryptPasswordEncoder para guardar um "hash" da senha no banco de dados, em vez do texto plano. A sua camada de serviço (ProfessorServiceImpl, EstudanteServiceImpl) deve ser responsável por criptografar a senha antes de salvar um novo utilizador.
  - **Criar Endpoint de Login:** Desenvolver um endpoint POST /api/login que receba um email e senha, verifique as credenciais contra os dados do banco e, em caso de sucesso, retorne um token de autenticação (JWT é o padrão moderno).
  - **Proteger Endpoints:** Configurar o Spring Security para que endpoints como /api/professores só possam ser acedidos se o utilizador enviar um token válido.

2. Implementar a Lógica de Avaliação (Requisito RF06)

- **O que falta:** A capacidade de o professor efetivamente salvar as notas de cada critério para um trabalho.
- **Ações:**
  - **Criar Tabela nota_criterio:** Adicionar ao seu script - bd1.sql uma nova tabela para armazenar a nota que um aluno (id_usuario) recebeu para um critério específico (id_criterio) de um trabalho (pert_aval_cod).
  - **Criar DAO, Service e Controller:** Desenvolver todo o fluxo para que o back-end possa receber e salvar um conjunto de notas para um trabalho. O AvaliarTrabalhoPage.js no front-end irá consumir este novo endpoint.

3. Implementar o Upload de Ficheiros (Requisito RF03)

- **O que falta:** O back-end ainda não sabe como receber um ficheiro.
- **Ações:**
  - Modificar o TrabalhoController para ter um endpoint que aceite requisições

do tipo multipart/form-data.
    - A anotação a ser usada no método do controller será @RequestParam("file") MultipartFile file.
    - O TrabalhoService receberá este ficheiro, irá salvá-lo num diretório no servidor e guardará o caminho para o ficheiro na coluna arquivo_url da tabela Trabalho.

4. Criar Endpoints para os Relatórios (Requisitos RF07 e RF08)

- **O que falta:** Os endpoints que o front-end irá chamar para obter os dados dos relatórios.
- **Ações:**
    - **Criar Novos Métodos no DAO:** Adicionar métodos que executem as consultas SQL complexas necessárias para os relatórios (usando JOIN, GROUP BY, AVG(), etc.).
    - **Criar Novos Métodos no Service:** Orquestrar a chamada a esses novos métodos do DAO.
    - **Criar Novos Endpoints no Controller:** Expor os dados dos relatórios em URLs como /api/relatorios/individual/{trabalhoId} e /api/relatorios/geral/{avaliacaoId}.

✅ **Tarefas Críticas de Front-End**

Estas são as tarefas para transformar o esqueleto funcional do React numa aplicação completa e interativa.

1. Integrar com a API Real

- **O que falta:** Substituir todas as simulações por chamadas reais.
- **Ações:**
    - **Login e Registo:** Conectar os formulários de LoginPage e RegisterPage aos endpoints reais de segurança e criação de utilizadores.
    - **Upload:** Fazer a EnviarTrabalhoPage enviar o ficheiro de verdade para o endpoint de upload.
    - **Relatórios:** Fazer as páginas de relatório (RelatorioGeralPage, RelatorioIndividualPage) buscarem os dados dos novos endpoints de relatório do back-end.

2. Implementar os Componentes de Gráfico (Requisitos RF07 e RF08)

- **O que falta:** A visualização dos dados dos relatórios.
- **Ações:**
    - **GraficoRadar.js:** Integrar este componente na RelatorioIndividualPage, passando os dados reais das notas por critério.

    - **HistogramaNotas.js:** Criar este novo componente (usando Chart.js) e integrá-lo na RelatorioGeralPage para mostrar a distribuição das notas finais.

3. Gestão de Estado e Feedback ao Utilizador

- **O que falta:** Uma experiência de utilizador mais polida.
- **Ações:**
    - **Loading States:** Mostrar um indicador de "a carregar..." (como o LoadingSpinner.js que planeámos) sempre que uma chamada à API estiver em andamento.
    - **Mensagens de Erro/Sucesso:** Criar um componente de notificação para dar feedback claro ao utilizador (ex: "Disciplina criada com sucesso!", "Erro: este email já existe.").

4. Estilização Final

- **O que falta:** A aparência visual da aplicação.
- **Ações:**
    - Aplicar CSS ou usar uma biblioteca de componentes (como Material-UI ou Ant Design) para estilizar todas as páginas, formulários, botões e tabelas, garantindo uma interface limpa e profissional.

Conclusão

O seu projeto tem uma base excelente. Para a entrega final, o foco deve ser na **segurança**, na **implementação das funcionalidades-chave** (avaliação e upload) e, principalmente, na **criação e visualização dos relatórios**, que são um diferencial do seu trabalho.